

Data Structures

Lecture 6: Linked List

Soobin Um

Mar. 23, 2026

Announcements

코딩테스트 관련 수정 공지

일정(소용대 학과 주최):

- PCCE: 5/21(목) 오후 6시 @ 오프라인
- PCCP: 5/23(토) 오후 2시 @ 온라인

사용 언어: C++ / Python / Java 중 택 1

성적 반영: 코딩테스트만 별도 10%

평가 기준: 기준

(40%) 중간고사

(40%) 기말고사

(10%) 과제

(10%) 출석 + 수업참여도(PCCE/P 응시 결과)

평가 기준: 수정

(35%) 중간고사

(35%) 기말고사

(10%) 과제

(10%) 출석

(10%) 수업참여도(PCCE/P 응시 결과)

다음 수업 휴강 공지

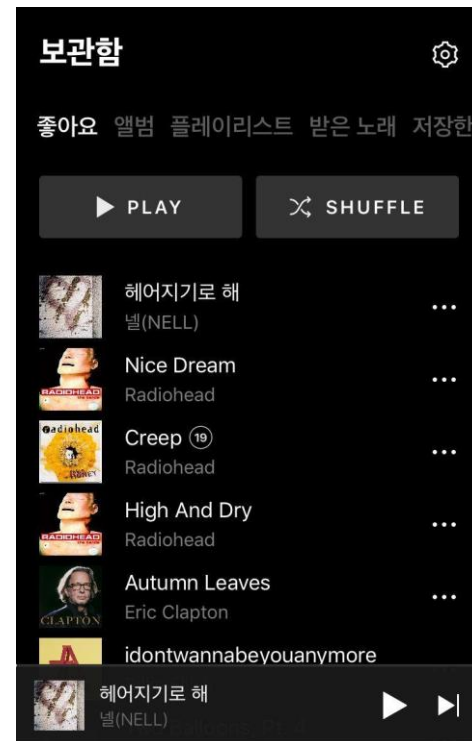
3/25(수) 수업 휴강

→ 온라인 영상 게시 예정

Recap

Lists (리스트)

- 순서가 있는 데이터의 나열(ordered collection)
- 리스트를 구성하는 각 데이터(원소)는 리스트 상에서 각자 자신의 위치(인덱스)를 가짐
 - 인덱스 기반 접근 가능 (예: list[0], list[1])



Key Operations

- **탐색 (Search)**
 - 특정 인덱스의 원소를 찾거나, 특정 값을 가진 원소를 찾는 것
- **갱신 (Update)**
 - 특정 인덱스의 원소를 새로운 값으로 교체
- **삽입 (Insert)**
 - 새로운 원소를 원하는 위치에 추가
 - 배열에서의 경우, 뒤의 원소들을 밀어내야 함
- **삭제 (Delete)**
 - 특정 위치의 원소를 제거
 - 배열에서의 경우, 뒤의 원소들을 앞으로 당겨야 함
- **순회 (Traverse)**
 - 리스트의 모든 원소를 차례대로 읽음
 - 예: 출력, 합계 계산

List ADT

- 구현 방법과 상관없이 인터페이스 차원에서 “**위치를 가진 원소들의 유한 순서열**”을 정의

```
public interface List<E> {  
    public void clear();  
    public void insert(int pos, E item);  
    public void append(E item);  
    public void update(int pos, E item);  
    public E getValue(int pos);  
    public E remove(int pos);  
    public void printAll();  
}
```

※ 제네릭 타입 E: Element의 약자로, 리스트에 어떤 자료형을 담을지를 **나중에 결정**

List ADT: 연산 동작 예

- List L = <12, 32, 15>
- L.insert(1, 99) $\Rightarrow$ <12, 99, 32, 15>
- L.remove(3) $\Rightarrow$ <12, 99, 32>
- L.update(0, 61) $\Rightarrow$ <61, 99, 32>
- L.length() $\Rightarrow$ 3
- L.clear() $\Rightarrow$ <>

```
public interface List<E> {  
    public void clear();  
    public void insert(int pos, E item);  
    public void append(E item);  
    public void update(int pos, E item);  
    public E getValue(int pos);  
    public E remove(int pos);  
    public void printAll();  
}
```

Question: ADT → Data Structure

“리스트를 실제로 어떻게 구현할 수 있을까?”
(= 리스트 상에서의 원소들을 어떻게 줄 세울 수 있을까?)

배열(array) 기반 구현

: 연속된 빈칸에 데이터를 차례대로 저장

연결 리스트(linked list) 기반 구현

: 각 칸마다 옆 칸을 가리키는 화살표를 붙여서 연결

배열 기반 리스트 구현

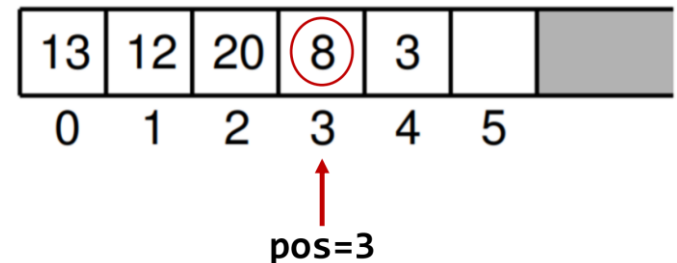
- 배열과 사이즈(size) 변수를 통해 리스트 구현
 - 배열: 실제 원소를 저장하는 고정 크기 공간
 - 사이즈 변수: 현재 배열에 저장된 원소 개수 관리

```
class ArrayListDemo {  
    private int[] arr();  
    private int size;  
    public ArrayListDemo(int capacity) {  
        arr = new int[capacity];  
        size = 0;  
    }  
}
```

탐색 Search

- 인덱스 접근을 통해 특정 위치의 원소를 빠르게 탐색 가능($\in O(1)$)
 - 단, 특정 값 찾기를 위해선, 선형 탐색이 필요

```
public int getValue(int pos) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    return arr[pos];  
}
```

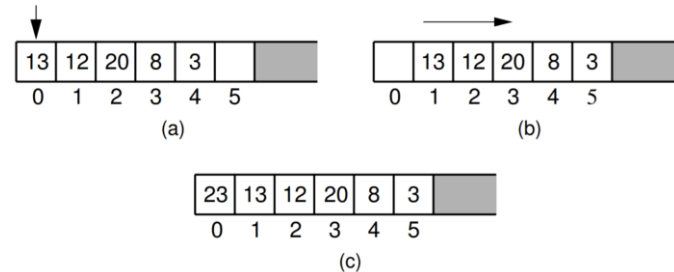


삽입 Insert

- 특정 위치에 새로운 값을 넣기 위해, 뒤에 있는 원소들을 한 칸씩 뒤로 이동($\in O(n)$)

```
public void insert(int pos, int item) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    for (int i = size; i > pos; i--)  
        arr[i] = arr[i-1];  
  
    arr[pos] = item;  
    size++;  
}
```

Insert 23:



Pros and Cons

- **장점**

- 특정 위치의 데이터로의 매우 빠른 접근 가능($\in O(1)$)
- 구현이 단순하고 직관적
- 메모리에 연속 저장되므로 캐시 친화적

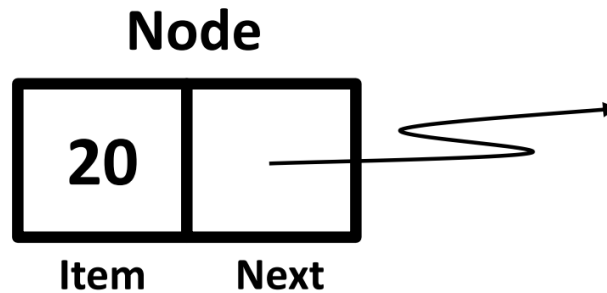
- **단점**

- 삽입/삭제 연산 시 많은 원소 이동 필요 ($\in O(n)$)
- 크기가 고정되어 있음
 - 넣으려는 데이터가 더 많을 경우, 에러 발생
 - 넣으려는 데이터가 너무 적을 경우, 메모리 낭비

Linked List

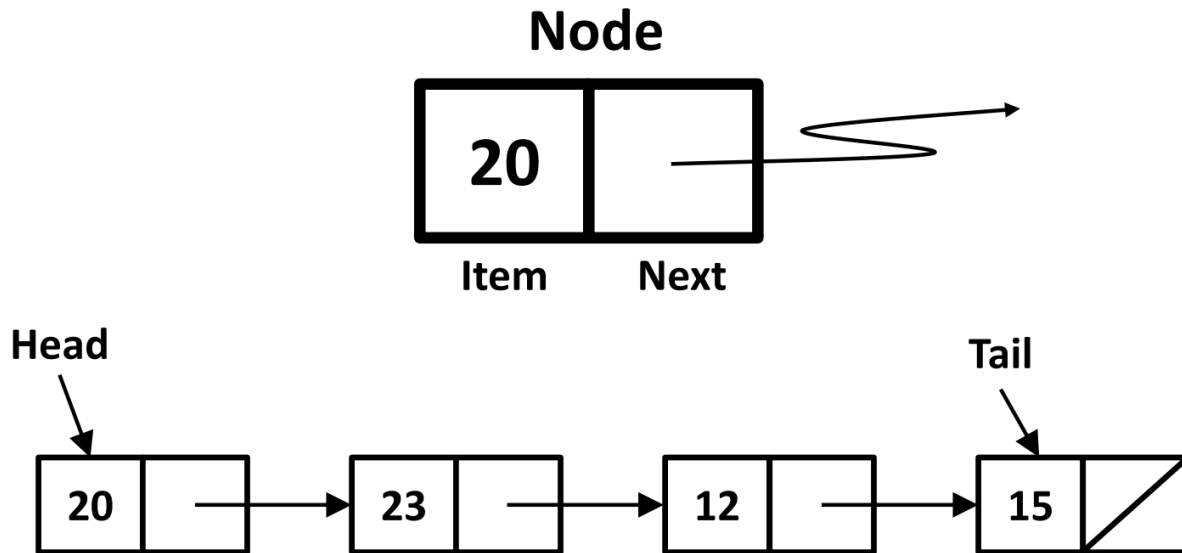
구조 및 특징

- **노드 (Node):** 연결 리스트 상에서 데이터를 저장하는 기본 단위
 - 데이터 필드 (item): 실제 원소 저장
 - 포인터 필드 (next): 다음 노드의 주소 저장



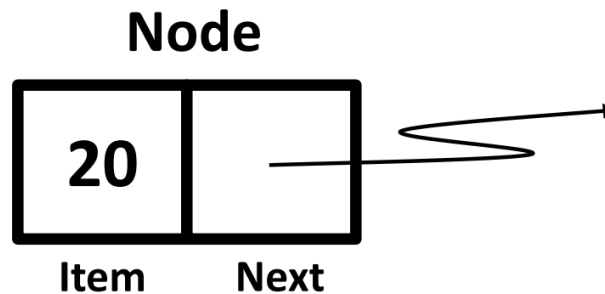
구조 및 특징

- **노드 (Node):** 연결 리스트 상에서 데이터를 저장하는 기본 단위
 - 데이터 필드 (item): 실제 원소 저장
 - 포인터 필드 (next): 다음 노드의 주소 저장
- **헤드 (Head) 포인터:** 연결 리스트의 시작을 알리는 포인터
- **사이즈 변수:** 현재 연결 리스트 상에 저장된 원소 개수 관리
- 배열과 달리, **메모리 연속성 불필요**



노드 클래스

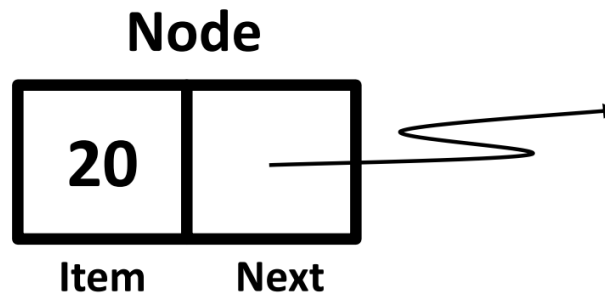
```
class Node<E> {  
    E item;  
    Node<E> next;  
    Node(E item, Node<E> next) {  
        this.item = item;  
        this.next = next;  
    }  
}
```



노드 클래스

```
class Node<E> {  
    E item;  
    Node<E> next;  
    Node(E item, Node<E> next) {  
        this.item = item;  
        this.next = next;  
    }  
}
```

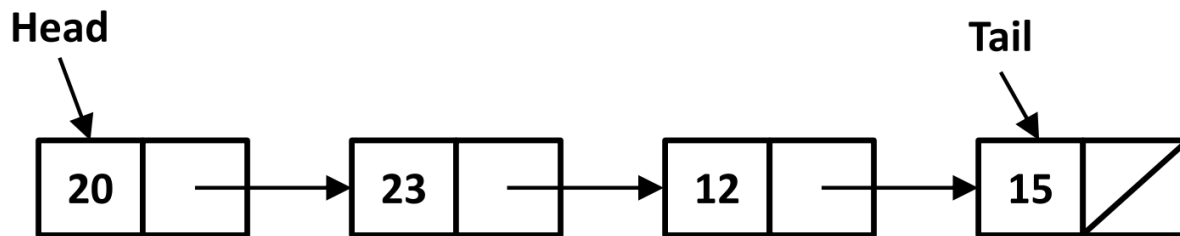
생성자: 객체가 만들어질 때 해당 클래스의 필드 값들을 초기화



노드(객체) 생성

```
Node<Integer> n15 = new Node<>(15, null);  
Node<Integer> n12 = new Node<>(12, n15);  
Node<Integer> n23 = new Node<>(23, n12);  
Node<Integer> n20 = new Node<>(20, n23);
```

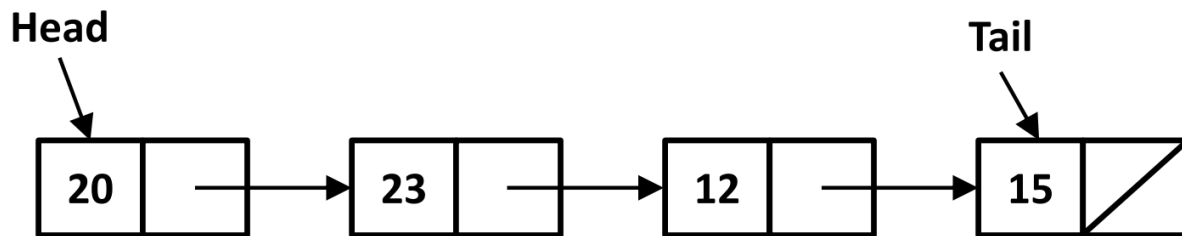
```
Node<Integer> head = n20;  
Node<Integer> tail = n15;
```



노드(객체) 생성

```
Node<Integer> n15 = new Node<>(15, null);  
Node<Integer> n12 = new Node<>(12, n15);  
Node<Integer> n23 = new Node<>(23, n12);  
Node<Integer> n20 = new Node<>(20, n23);
```

```
Node<Integer> head = n20;  
Node<Integer> tail = n15;
```



Q. 다음 코드를 실행했을 때 출력되는 값은?

```
System.out.println(head.next().next().item());
```

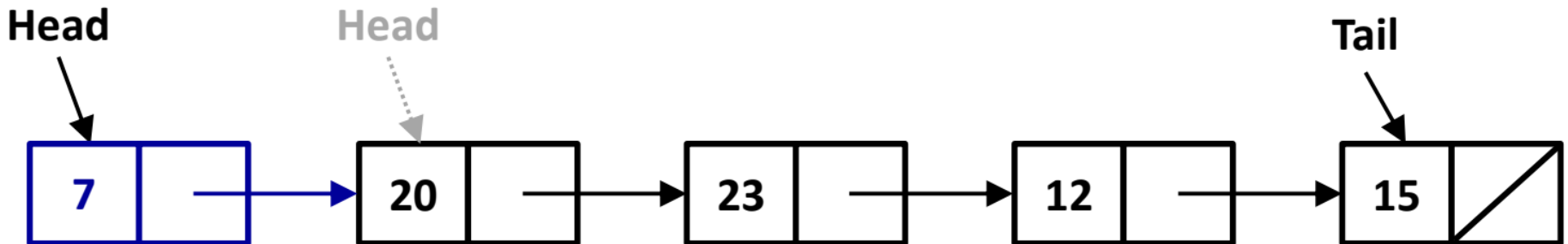
연결 리스트 클래스

```
public class LinkedList<E> {  
    /* LinkedList 클래스 내부에서만 접근 가능한 내부 클래스 */  
    private static class Node<E> {  
        E item;  
        Node<E> next;  
        Node(E item, Node<E> next) {  
            this.item = item;  
            this.next = next;  
        }  
    }  
    private Node<E> head; // 클래스 내부에서만 접근 가능  
    private Node<E> tail; // 클래스 내부에서만 접근 가능  
    private int size; // 클래스 내부에서만 접근 가능  
  
    public LinkedList() {  
        head = null;  
        tail = null;  
        size = 0;  
    }  
    // Operations  
}
```


삽입(1): 시작 위치에 노드 추가

- head 포인터만 연결하면 되므로 매우 효율적 ($\in O(1)$)

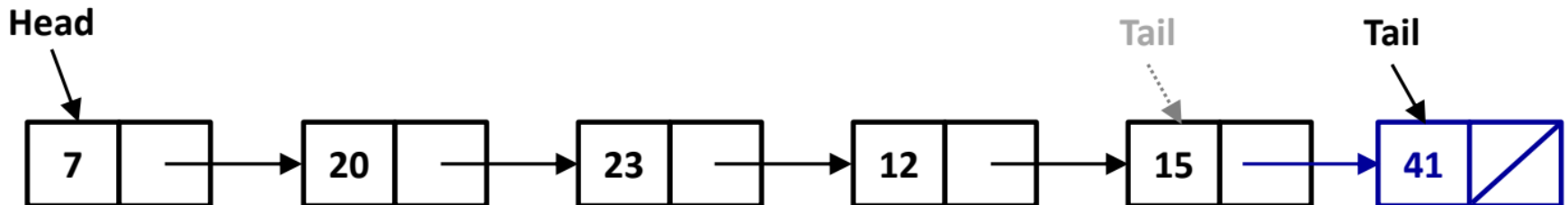
```
public void addFirst(E item) {  
    Node<E> newNode = new Node<>(item, head);  
    head = newNode;  
  
    if (tail == null)  
        tail = newNode;  
    size++;  
}
```



삽입(2): 마지막 위치에 노드 추가

- tail 포인터만 연결하면 되므로 매우 효율적 ($\in O(1)$)

```
public void addLast(E item) {  
    Node<E> newNode = new Node<>(item, null);  
  
    if (head == null) {  
        head = newNode;  
        tail = newNode;  
    } else {  
        tail.next = newNode;  
        tail = newNode;  
    }  
    size++;  
}
```



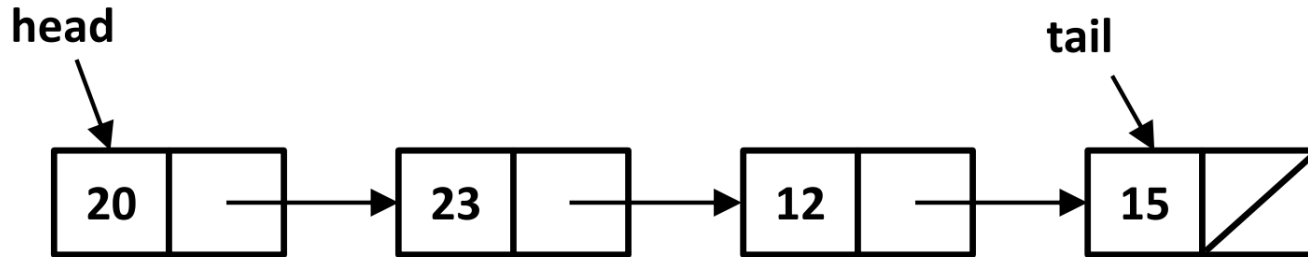
삽입(3): 지정된 위치에 노드 추가

- 이전 노드의 next를 새 노드로 연결.
- 연결 작업 자체는 효율적 $O(1)$, 하지만 접근하는 과정에서 $O(n)$

삽입(3): 지정된 위치에 노드 추가

- 이전 노드의 next를 새 노드로 연결.
- 연결 작업 자체는 효율적 $O(1)$, 하지만 접근하는 과정에서 $O(n)$

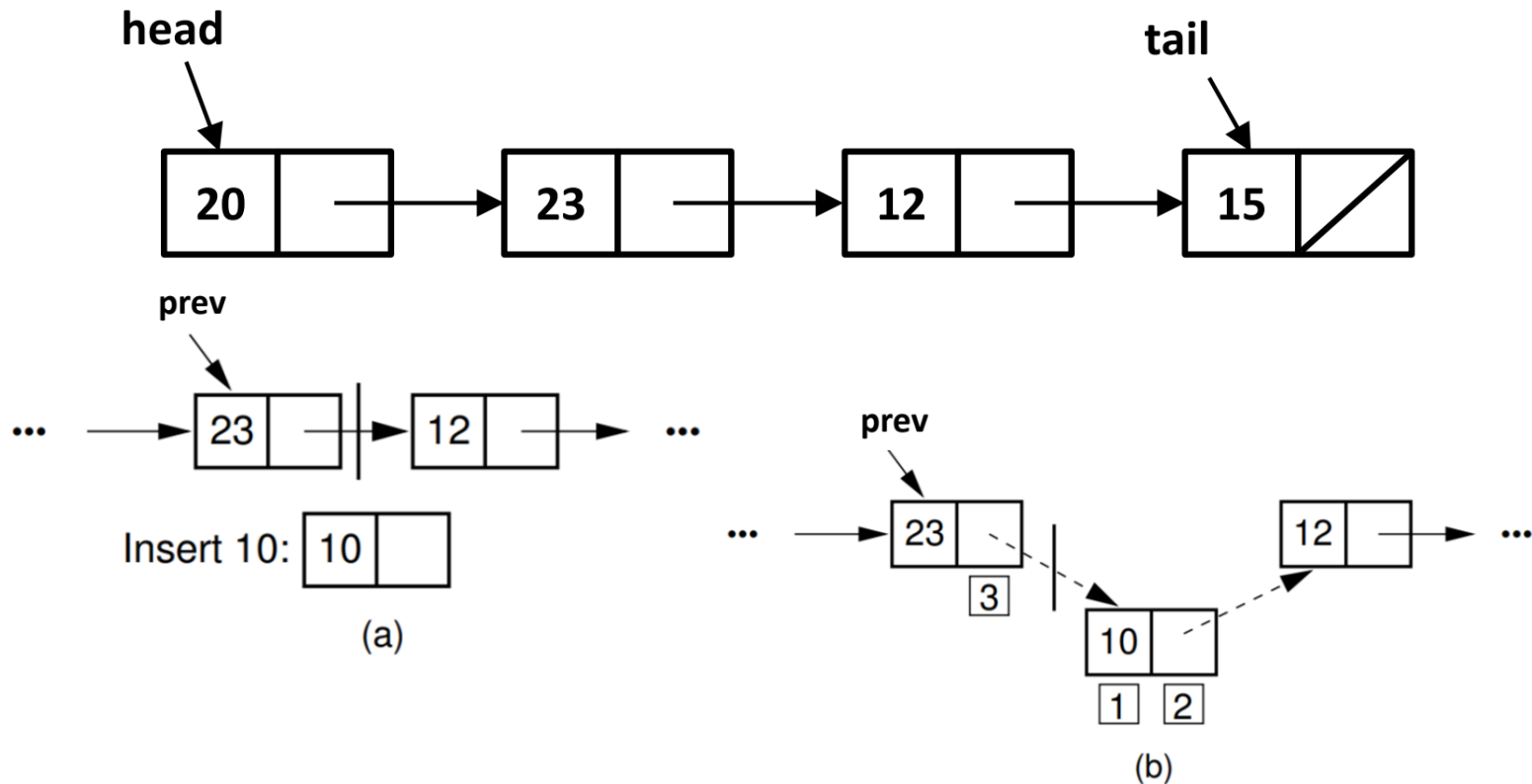
Task: pos=2 (즉, 23과 12 사이) 위치에 새로운 노드 10을 넣고자 하는 경우



삽입(3): 지정된 위치에 노드 추가

- 이전 노드의 next를 새 노드로 연결.
- 연결 작업 자체는 효율적 $O(1)$, 하지만 접근하는 과정이 $O(n)$

Task: pos=2 (즉, 23과 12 사이) 위치에 새로운 노드 10을 넣고자 하는 경우



삽입(3): 지정된 위치에 노드 추가

```
public void insert(int pos, E item) {
    if (pos < 0 || pos > size) throw new IndexOutOfBoundsException();

    if (pos == 0) {
        addFirst(item);
        return;
    } else if (pos == size) {
        addLast(item);
        return;
    }

    Node<E> prev = head;
    for (int i=0; i<pos-1; i++)
        prev = prev.next; // 바로 앞 노드까지 이동
    prev.next = new Node<>(item, prev.next);

    size++;
}
```

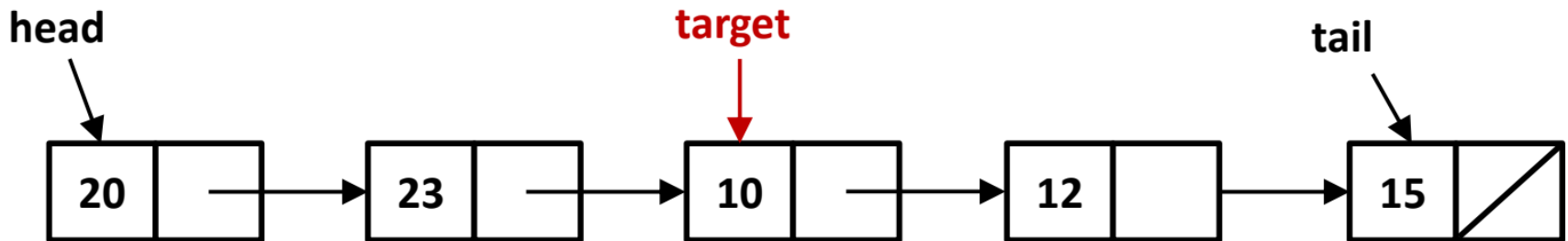
삭제: 지정된 위치의 노드 제거

- 삭제할 노드의 이전 노드까지의 접근 필요
- 삭제 작업 자체는 효율적 $O(1)$, 하지만 접근하는 과정이 $O(n)$

삭제: 지정된 위치의 노드 제거

- 삭제할 노드의 이전 노드까지의 접근 필요
- 삭제 작업 자체는 효율적 $O(1)$, 하지만 접근하는 과정이 $O(n)$

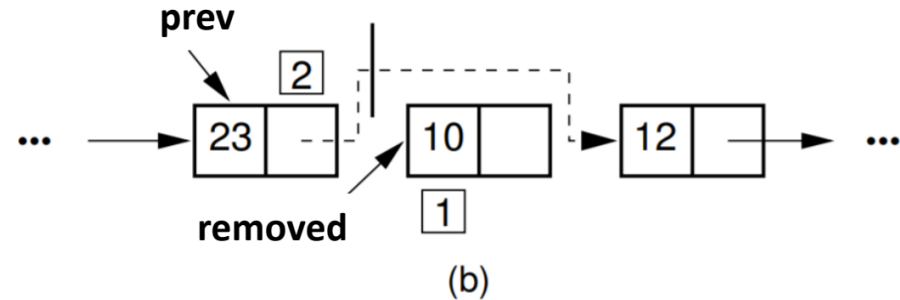
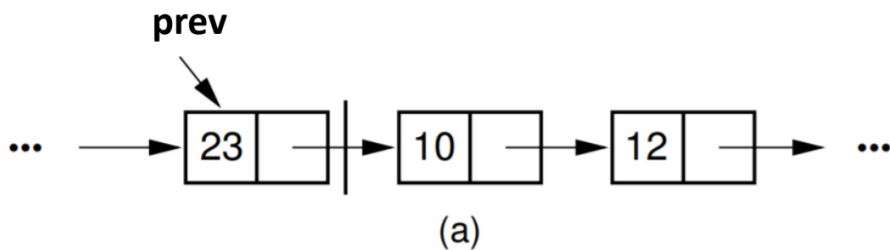
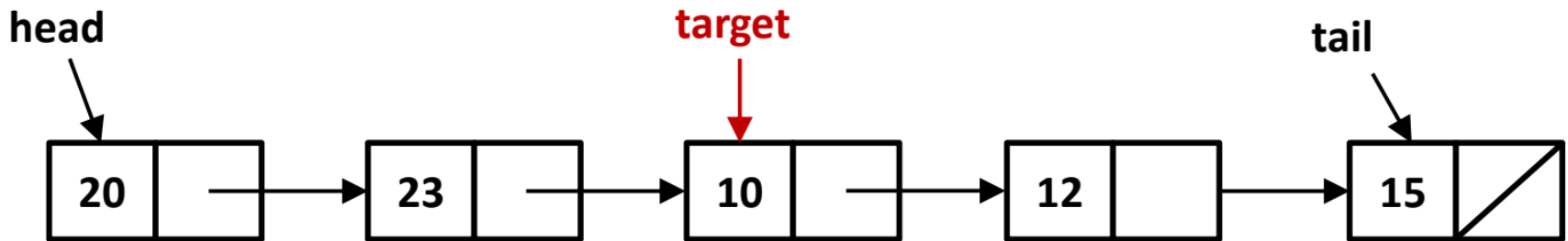
Task: pos=2 위치에 있는 노드 10을 삭제하고자 함



삭제: 지정된 위치의 노드 제거

- 삭제할 노드의 이전 노드까지의 접근 필요
- 삭제 작업 자체는 효율적 $O(1)$, 하지만 접근하는 과정이 $O(n)$

Task: pos=2 위치에 있는 노드 10을 삭제하고자 함



삭제: 지정된 위치의 노드 제거

```
public E remove(int pos) {
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();

    E removed;
    if (pos == 0) { // 맨 앞 삭제
        removed = head.item;
        head = head.next;
        if (head == null)
            tail = null;
    } else {
        Node<E> prev = head;
        for (int i=0; i<pos-1; i++)
            prev = prev.next; // 바로 앞 노드까지 이동
        Node<E> target = prev.next;
        removed = target.item;
        prev.next = target.next;

        if (target == tail) // 맨 끝 삭제하게 된 경우
            tail = prev;
    }
    size--;
    return removed;
}
```

탐색

- 탐색: 맨 앞에서부터 차례대로 따라가야 함 ($\in O(n)$)

```
public E getValue(int pos) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    Node<E> cur = head;  
    for (int i=0; i<pos; i++)  
        cur = cur.next; // 해당 노드까지 이동  
  
    return cur.item;  
}
```

탐색, 사이즈 출력

- 탐색: 맨 앞에서부터 차례대로 따라가야 함 ($\in O(n)$)

```
public E getValue(int pos) {  
    if (pos < 0 || pos >= size) throw new IndexOutOfBoundsException();  
  
    Node<E> cur = head;  
    for (int i=0; i<pos; i++)  
        cur = cur.next; // 해당 노드까지 이동  
  
    return cur.item;  
}
```

- 사이즈 출력

```
public int size() {  
    return size;  
}
```

연결 리스트 사용 예제: main()

```
public static void main(String[] args) {  
    LinkedList<Integer> list = new LinkedList<>();  
  
    list.addLast(10);  
    list.addLast(20);  
    list.addLast(30);  
    list.addLast(40); // [10 → 20 → 30 → 40]  
  
    System.out.println("Removed value: " + list.remove(3)); // 40  
    System.out.println("Removed value: " + list.remove(0)); // 10  
  
    System.out.println("Tail: " + list.getValue(list.size()-1)); // 30  
}
```

배열 기반 리스트 vs. 연결 리스트

연산	배열 기반 리스트	연결 리스트	참고사항
탐색 (인덱스 접근)	$O(1)$	$O(n)$	배열은 인덱스로 즉시 접근, 연결 리스트는 순차 탐색 필요
탐색 (특정 값)	$O(n)$	$O(n)$	둘 다 순차 탐색 필요
삽입 (마지막 위치)	$O(1)$	$O(1)$	배열은 공간 여유 시 $O(1)$, 연결 리스트는 tail 사용 시 $O(1)$
삽입 (지정 위치)	$O(n)$	$O(n)$ (접근) + $O(1)$ (링크 변경)	배열은 원소 이동 필요, 연결 리스트는 이전 노드까지 탐색
삭제 (마지막 위치)	$O(1)$	$O(n)$	배열은 사이즈만 줄이면 됨, 연결 리스트는 tail 이전 노드 탐색 필요
삭제 (지정 위치)	$O(n)$	$O(n)$ (접근) + $O(1)$ (링크 변경)	연결 리스트 삭제 연산 자체는 $O(1)$, but 탐색은 $O(n)$
순회	$O(n)$	$O(n)$	둘 다 순차 탐색 필요

Pros and Cons

- **장점**

- **동적 크기**: 필요할 때마다 노드를 추가하면 되므로, **크기 제한이 없음**
- **삽입/삭제 용이**
 - 포인터만 변경하면 됨 ($O(1)$, 단, **접근까진 $O(n)$** 정도의 비용 발생)
 - 맨 앞 삽입/삭제는 항상 $O(1)$
- **메모리 활용 유연**
 - 메모리가 연속적이지 않아도 되므로, **흩어진 공간 활용 가능**

Pros and Cons

• 장점

- 동적 크기: 필요할 때마다 노드를 추가하면 되므로, 크기 제한이 없음
- 삽입/삭제 용이
 - 포인터만 변경하면 됨 ($O(1)$, 단, 접근까진 $O(n)$ 정도의 비용 발생)
 - 맨 앞 삽입/삭제는 항상 $O(1)$
- 메모리 활용 유연
 - 메모리가 연속적이지 않아도 되므로, 흩어진 공간 활용 가능

• 단점

- 임의 접근 불가: 순차 탐색 필요 ($O(n)$)
- 추가 메모리 사용: 각 노드마다 데이터+포인터 저장 필요
- 구현 복잡성: 삽입/삭제 시 포인터 관리 필요
- 낮은 캐시 효율성
 - 배열은 연속된 메모리 구조여서 CPU 캐시 친화적인 반면, 연결 리스트는 흩어져 있어 캐시 적중률 낮음

Look ahead

스택, 큐(Stack and Queue)